

Tugas Praktikum Pertemuan 11

Nama : Muhamad Daud Ghifari

NIM : J0403251026

Kelas : TPL A

1. Praktikum 1 - Membuat Adjacency Matrix

Code:

```
16
17 def createGraph(V, edges):
18     # Membuat matriks V x V dengan nilai awal 0
19     mat = [[0 for _ in range(V)] for _ in range(V)]
20
21     # Mengisi matriks dengan nilai 1 untuk setiap edge
22     for it in edges:
23         u = it[0]
24         v = it[1]
25
26         mat[u][v] = 1
27         mat[v][u] = 1 # Karena graph tidak berarah
28
29     return mat
30
31 # Jalankan program
32
33 if __name__ == "__main__":
34     # karena 0,1,2,3 jadi ada 4 vertex
35     V = 4
36
37     # mengisi hubungan antar vertex
38     edges = [[0,1],[0,2],[1,2],[2,3]]
39
40     mat = createGraph(V, edges)
41     print("Representasi antar vertex:")
42     for i in range(V):
43         for j in range(V):
44             print(mat[i][j], end=" ")
45         print()
46
47
```

Hasil :

```
Representasi antar vertex:
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0
```

2. Praktikum 2 - Membuat Adjacency List Tugas

Code:

```
17 def createGraph(v, edges):
18     # buat dictionary
19     adj = {}
20
21     # mengisi dictionary sesuai hubungan
22     for it in edges:
23         u, v = it[0], it[1]
24
25         # inialisasi vertex u jika belum ada di dictionary
26         if u not in adj:
27             adj[u] = []
28         # menambahkan vertex v ke dalam list adjacency vertex u, begitu juga yang v ke u
29         adj[u].append(v)
30
31         if v not in adj:
32             adj[v] = []
33         adj[v].append(u)
34
35     return adj
36
37 if __name__ == "__main__":
38     # menginisiasi 4 vertex
39     v = 4
40
41     # mengisi hubungan antar vertex
42     edges = [["A","B"], ["A","C"], ["B","D"], ["C","D"]]
43
44
45     adj = createGraph(v, edges)
46     print("Representasi graph dalam adjacency list: ")
47     for i in adj:
48         print(f"{i} : {adj[i]}")
49
```

Hasil:

```
Representasi graph dalam adjacency list:
A : ['B', 'C']
B : ['A', 'D']
C : ['A', 'D']
D : ['B', 'C']
```

3. Praktikum 3 - Konversi Matrix ke List

Code:

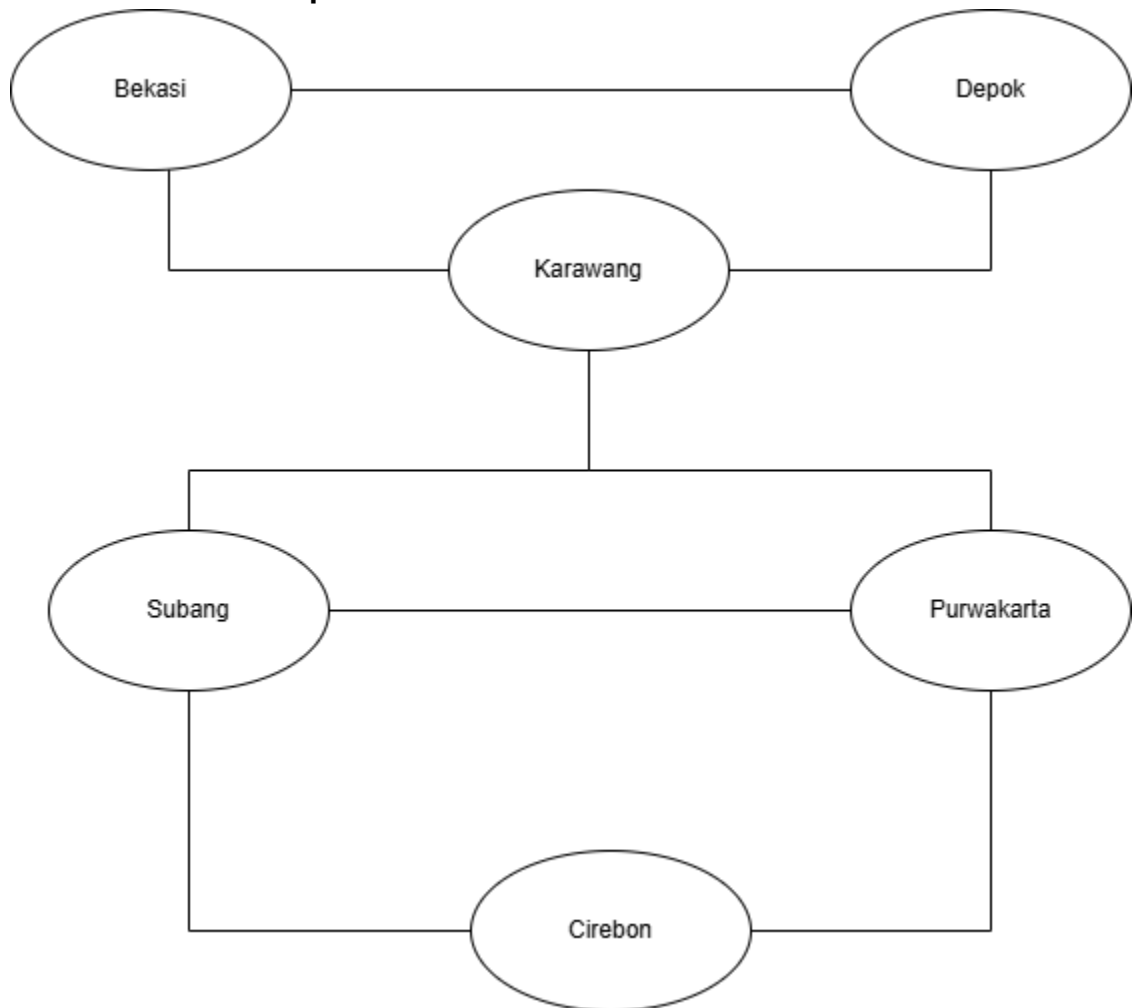
```
18
19 def konversi(matriks):
20     panjang_matriks = len(matriks)
21
22     # buat listnya berdasarkan panjang matriks
23     adj = [[] for _ in range(panjang_matriks)]
24
25     for i in range(panjang_matriks):
26         for j in range(panjang_matriks):
27             # mengirimkan nilai ke variabel adj jika nilai di dalam matriks adalah 1
28             if matriks[i][j] == 1:
29                 adj[i].append(j)
30     return adj
31
32
33 if __name__ == "__main__":
34     matriks = [[0, 1, 1, 0], [1, 0, 1, 0], [1, 1, 0, 1], [0, 0, 1, 0]]
35     adj = konversi(matriks)
36     print("Representasi matriks dalam bentuk Adjacency List: ")
37     for i in range(len(matriks)):
38         print(f"{i}:", end=" ")
39         for j in adj[i]:
40             print(f"{j}", end=" ")
41         print()
42
```

Hasil :

```
Representasi matriks dalam bentuk Adjacency List:
0: 1 2
1: 0 2
2: 0 1 3
3: 2
```

4. Tugas Praktikum 4 – Studi kasus Dunia nyata (Peta Kota)

Gambar Desain Graph:



Implementasi Dalam Python:

- **Adjacency List :**

Kode:

```
29 def adjacency_list(edges):
30     # buat dictionary
31     adj = {}
32     for it in edges:
33         u = it[0]
34         v = it[1]
35
36         # inisialisasi vertex u jika belum ada di dictionary
37         if u not in adj:
38             adj[u] = []
39         # menambahkan vertex v ke dalam list adjacency vertex u, begitu juga yang v ke u
40         adj[u].append(v)
41
42         if v not in adj:
43             adj[v] = []
44         adj[v].append(u)
45     return adj
46
```

```

61
62 # Adjacency List
63 # Membuat hubungan antar kota yang sesuai dengan gambar
64 edgesAdj = [
65     ["Bekasi", "Depok"],
66     ["Bekasi", "Karawang"],
67     ["Depok", "Karawang"],
68     ["Karawang", "Subang"],
69     ["Karawang", "Purwakarta"],
70     ["Subang", "Purwakarta"],
71     ["Subang", "Cirebon"],
72     ["Purwakarta", "Cirebon"],
73 ]
74 adj = adjacency_list(edgesAdj)
75 print("=====\n")
76 print("Representasi Rute jalan antar kota dalam bentuk List")
77
78 for i in adj:
79     print(f"{i} : {adj[i]}")
80

```

Hasil :

```

Representasi Rute jalan antar kota dalam bentuk List
Bekasi : ['Depok', 'Karawang']
Depok : ['Bekasi', 'Karawang']
Karawang : ['Bekasi', 'Depok', 'Subang', 'Purwakarta']
Subang : ['Karawang', 'Purwakarta', 'Cirebon']
Purwakarta : ['Karawang', 'Subang', 'Cirebon']
Cirebon : ['Subang', 'Purwakarta']
PS C:\laragon\www\Algoritma-StrukturData>

```

- Adjacency Matrix:

Kode :

```

16
17 def adjacency_matrix(V, edges):
18     mat = [[0 for _ in range(V)] for _ in range(V)]
19
20     for it in edges:
21         u = it[0]
22         v = it[1]
23
24         mat[u][v] = 1
25         mat[v][u] = 1 # graph nya tidak berarah, rutanya dua arah
26     return mat
27

```

```

47
48 if __name__ == "__main__":
49     # karena ada 6 kota, maka vertex nya 6
50     V = 6
51     # (Bekasi, Depok, Karawang, Subang, Purwakarta, Cirebon)
52     # mengisi hubungan antar kota
53     edges = [[0, 1], [0, 2], [1, 2], [2, 3], [2, 4], [3, 4], [3, 5], [4, 5]]
54
55     mat = adjacency_matrix(V, edges)
56     print("Representasi Rute jalan antar kota dalam bentuk Matriks:")
57     for i in range(V):
58         for j in range(V):
59             print(mat[i][j], end=" ")
60         print()
61

```

Hasil :

```
Representasi Rute jalan antar kota dalam bentuk Matriks:  
0 1 1 0 0 0  
1 0 1 0 0 0  
1 1 0 1 1 0  
0 0 1 0 1 1  
0 0 1 1 0 1  
0 0 0 1 1 0  
=====
```

Analisis Singkat

- **Jenis Graph**
Jenis Graph Yang digunakan adalah Undirected graph karena jalan antarkota dapat dilalui melalui kota a ke kota b maupun sebaliknya.
- **Alasan Memilih edge**
Saya memilih edge tersebut karena rute antarkotanya 2 arah/ undirected sehingga lebih mudah dianalisis.
- **Representasi mana yang lebih cocok**
Menurut saya representasi List yang lebih cocok, karena datanya sedikit sehingga memori yang digunakan lebih sedikit. Saya pribadi lebih suka membaca dalam bentuk list.
- **apakah graph termasuk dense atau sparse.**
Graph ini termasuk graph sparse karena edgenya sangat sedikit jika dibandingkan total kemungkinan hubungan/edge

Kesimpulan

Berdasarkan analisis, dapat disimpulkan bahwa jaringan hubungan antarkota ini dimodelkan sebagai Undirected Graph karena setiap jalan yang ada bersifat dua arah, sehingga memudahkan proses analisis rute. Penggunaan Adjacency List dinilai sebagai representasi yang tepat karena lebih hemat memori untuk jumlah data yang terbatas dan lebih enak dilihat. Selain itu, graf ini dikategorikan sebagai Sparse Graph karena jumlah koneksi yang ada (8 edge) jauh lebih sedikit dibandingkan total kemungkinan koneksi maksimalnya.